Stella Mary's College of Engineering

(Autonomous)

Aruthenganvillai, Azhikal Post–629202

Department of Artificial Intelligence and Data Science



24AD3351-Design and Analysis of Algorithm Lab Observation

NAME :

ROLL NUMBER :

REGISTERNUMBER:

YEAR&SEMESTER:

DEPARTMENT :

TABLEOFCONTENTS

EX NO	DATE	EXPERIMENTNAME	CO	MARKS	SIGNATURE

Ex No: 1 IMPLEMENTATION OF STRING MATCHING USING BRUTE FORCE APPROACH

Date:

Aim:

To write a python program to implement String Matching using Brute Force approach.

Algorithm:

1. Start
2. Get an input String and Search pattern as input.
3. Find the length of each string to iterate over strings.
4. Check the search pattern with the input string character by character.
5. If all the characters present in the search pattern matches with the input String, then return the position of the string
6. Continue step: 4 till the end of the input string is reached.
7. Display all the position of the pattern matched with the input string.
8. Stop.

Program:

```
# Brute-force string search method
def search(pat, txt):
    M = len(pat)
    N = len(txt)
    # A loop to slide pat[] one by one
    for i in range(N - M + 1):
        j = 0
        # For current index i, check for pattern match
        while(j < M):
            if (txt[i + j] != pat[j]):
```

```
                    break
            j += 1
        if (j == M):
            print("Pattern found at index ", i)

txt = input("Enter Input String: ")
pat = input("Enter search pattern: ")
search(pat, txt)
```

Result:

Thus, the above program to implement String Matching using Brute Force approach was done and executed successfully.

Ex No: 2 IMPLEMENTATION OF MULTIPLICATION OF LARGE INTEGERS USING DIVIDE AND CONQUER APPROACH

Date:

Aim:

To write a python program to implement multiplication of large integers using Divide and Conquer Approach.

Algorithm:

1. Start with two large integers, x and y.
2. Convert both x and y to strings for easier manipulation.
3. Check if the length of x or y is 1. If either is 1, return their product as the result.
4. Make sure both x and y have the same length by adding leading zeros if necessary.
5. Split the Numbers:
   - Determine n, the length of the strings x and y.
   - Calculate n_half as n // 2.
   - Split the strings x and y into two halves, a and b, and c and d, where each half has approximately half the length of the original string.
   - a = x[:n_half]
   - b = x[n_half:]
   - c = y[:n_half]
   - d = y[n_half:]
6. Recursively compute three products:
   - ac = karatsuba(a, c)
   - bd = karatsuba(b, d)
   - ad_bc = karatsuba((a + b), (c + d)) - ac - bd
7. The final result is computed as follows:
   - result = (10^(2 * n_half)) * ac + (10^n_half) * ad_bc + bd
   - Where, n_half is half the length of the original numbers, and (10^(2 * n_half)) and (10^n_half) are used to shift the products to the correct positions.
8. Return the result as the final product of x and y.

Program:

```
def karatsuba(x, y):
    x_str, y_str = str(x), str(y)      #Convert input numbers to strings for easy
manipulation
    if len(x_str) == 1 or len(y_str) == 1:  #If the input numbers have only one digit
        return x * y
    # padding with leading zeros to make both numbers have the same length
    max_len = max(len(x_str), len(y_str))
    x_str = x_str.zfill(max_len)
    y_str = y_str.zfill(max_len)
    # Split the input numbers into halves
    n = len(x_str)
    n_half = n // 2
    a, b = int(x_str[:n_half]), int(x_str[n_half:])
    c, d = int(y_str[:n_half]), int(y_str[n_half:])
    # Recursively compute the three products
    ac = karatsuba(a, c)
    bd = karatsuba(b, d)
    ad_bc = karatsuba((a + b), (c + d)) - ac - bd
    # Combine the three products to get the final result
    result = (10**(2 * n_half )) * ac + (10**n_half ) * ad_bc + bd
    return result
#Main Part
num1 = int(input("Enter Number 1: "))
num2 = int(input("Enter Number 2: "))
result = karatsuba(num1, num2)
print(result)
```

Result:

Thus, the above program to implement multiplication of large integers using Divide and Conquer Approach was done and executed successfully.

Ex No: 3 IMPLEMENTATION OF TOPOLOGICAL SORTING USING DECREASE AND CONQUER TECHNIQUE

Date:

Aim:

To write a python program to implement Topological sorting using Decrease and Conquer Technique.

Algorithm:

1. Import defaultdict
2. Create a class graph
3. Create a function edge( )
4. Create a function visit( )
5. create a function topological_sort( )
6. Identify vertices that have no incoming edges.
7. Select that vertex as starting vertex of a graph
8. Delete the starting vertex or the vertex with no incoming edges and delete all its outgoing edges from the graph.
9. Place the deleted vertex in the output list.
10. Repeat until the graph is empty
11. The graph is now sorted.
12. Print the result

Program:

```
from collections import defaultdict
class Graph:
    def __init__(self, directed=False):
        self.graph = defaultdict(list)
        self.directed = directed
    def Edge(self, frm, to):
        self.graph[frm].append(to)
```

```
                if self.directed is False:
                        self.graph[to].append(frm)
                else:
                        self.graph[to] = self.graph[to]
        def visit(self, s, visited, sortlist):
                visited[s] = True
                for i in self.graph[s]:
                        if not visited[i]:
                                self.visit(i, visited, sortlist)
                sortlist.insert(0, s)
        def topological_Sort(self):
                visited = {i: False for i in self.graph}
                sortlist = [ ]
                for v in self.graph:
                        if not visited[v]:
                                self.visit(v, visited, sortlist)
                print(sortlist)
g = Graph(directed=True)
g.Edge(1, 2)
g.Edge(1, 3)
g.Edge(3, 4)
g.Edge(4, 5)
g.Edge(5, 6)
g.Edge(6, 7)
g.Edge(7, 8)
print("The Result after Topological Sort:")
g.topological_Sort( )
```

Result:

Thus, the above program to implement Topological sorting using Decrease and Conquer Technique was done and executed successfully.

Ex No: 4 IMPLEMENTATION OF HEAP SORT USING TRANSFORM AND CONQUER TECHNIQUE

Date:

Aim:

To write a python program to implement Heap Sort using Transform and Conquer technique.

Algorithm:

1. Define a function heapify( )
2. Pass three parameters array, a and b
3. Find largest among root and children
4. If the root is not the largest, swap it with children
5. Continue to heapify
6. Define function Heap_Sort( )
7. Pass array as a parameter
8. Build Max-heap and swap
9. Heapify the root element
10. Display the result in sorted order.

Program:

```
def heapify(array, a, b):
    largest = b
    l = 2 * b + 1
    root = 2 * b + 2
    if ( l < a and array[b] < array[l]):
        largest = l
    if (root < a and array[largest] < array[root]):
```

```
            largest = root
    # Change root
    if (largest != b):
       array[b], array[largest] = array[largest], array[b]
       heapify(array, a, largest)
def Heap_Sort(array):
    a = len(array)
    for b in range(a // 2 - 1, -1, -1):
            heapify(array, a, b)
    for b in range(a-1, 0, -1):
            array[b], array[0] = array[0], array[b] # swap
            heapify(array, b, 0)


array= [ ]
n=int(input("Enter array size: "))
print("Enter array elements:")
for i in range(0,n):
    a=int(input( ))
    array.append(a)
print("The original array is: ", array)
Heap_Sort(array)
print ("Array after sorting Array is: ", array)
```

Result:

Thus, the above program to implement Heap Sort using Transform and Conquer technique was done and executed successfully.

Ex No: 5 IMPLEMENTATION OF WARSHALL'S AND FLOYD 'S ALGORITHMS USING DYNAMIC PROGRAMMING METHOD

Aim:

To write a python program to implement Warshall's and Floyd 's algorithms Dynamic programming method.

Algorithm:

1. Create a matrix A^0 of dimension n*n where n is the number of vertices. The row and the column are indexed as i and j respectively. i and j are the vertices of the graph.
2. Each cell A[i][j] is filled with the distance from the ith vertex to the jth vertex. If there is no path from i^{th} vertex to j^{th} vertex, the cell is left as infinity.
3. Now, create a matrix A^1 using matrix A^0. Using the intermediate vertex find the the shortest distance by, A[i][j] = min(A[i][j], A[i][k] + A[k][j])
4. Step 3 is repeated until all the vertices are visited and updated.
5. Finally using A^n Matrix gives the shortest path between each pair of vertices.
6. Display the matrix to get the result.

Program:

```
nv = 4

INF = 999

def floyd(G):

    dist = list(map(lambda i: list(map(lambda j: j, i)), G))

    # Adding vertices individually

    for k in range(nv):

        for i in range(nv):

            for j in range(nv):
```

```
                    dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j])
        display(dist)
# Printing the output
def display(dist):
        print("Shortest distance between every pair of vertices")
        for i in range(nv):
                for j in range(nv):
                        if(dist[i][j] == INF):
                                print("INF", end=" ")
                        else:
                                print(dist[i][j], end="  ")
                print(" ")
G = [  [0, 5, INF, 3],
        [50, 0, 15, 5],
        [30, INF, 0, 15],
        [15, INF, 5, 0]]
floyd(G)
```

Result:

Thus, the above program to implement Warshall's and Floyd's algorithm using Dynamic programming method was done and executed successfully.

Ex No: 6 IMPLEMENTATION OF DIJKSTRA'S ALGORITHM USING GREEDY TECHNIQUE

Aim:

To write a python program to implement Dijkstra's algorithm using Greedy Technique.

Algorithm:

1. Create a list of distances initialized to Infinity.
2. Initialize the distance of the root node to 0.
3. Create a list of visited nodes, all initialized to False.
4. Loop through the nodes and pick the next node to visit based on distance. If looped through all the available nodes and haven't found a valid one, break out of the inner loop.
5. Add the closest node to the list of visited nodes.
6. Set the distance of the visited node to the shortest distance available.
7. Finally return the list of distances.

Program:

```
class Graph( ):

    def __init__(self, vertices):

        self.V = vertices

        self.graph = [[0 for column in range(vertices)] for row in range(vertices)]


    def printSolution(self, dist):

        print("Vertex \t Distance from Source")

        for node in range(self.V):

            print(node, "\t\t", dist[node])
```

```
def minDistance(self, dist, sptSet):
        min = 100
        for i in range(self.V):
            if dist[i] < min and sptSet[i] == False:
                min = dist[i]
                min_index = i
        return min_index


def dijkstra(self, src):
        dist = [100] * self.V
        dist[src] = 0
        sptSet = [False] * self.V
        for cout in range(self.V):
            u = self.minDistance(dist, sptSet)
            sptSet[u] = True
            for i in range(self.V):
                if (self.graph[u][i] > 0 and
                sptSet[i] == False and
                dist[i] > dist[u] + self.graph[u][i]):
                    dist[i] = dist[u] + self.graph[u][i]
        self.printSolution(dist)
```

```
g = Graph(5)
g.graph = [[0, 1, 0, 0,0],
           [1,0, 2, 3,0],
           [0, 2, 0,1,5],
           [0, 3,7, 0, 1],
         [0,0,5,1,0]
        ]
g.dijkstra(0)
```

Result:

Thus, the above program to implement Dijkstra's algorithm using Greedy Technique was done and executed successfully.

Ex No: 7 IMPLEMENTATION OF N-QUEEN'S PROBLEM USING BACKTRACKING METHOD

Aim:

To write a python program to implement N-Queen problem using Backtracking method.

Algorithm:

1. Start
2. Get the number of queens to be placed as n
3. Initialize the chess board as a Matrix.
4. Place the queen in their corresponding place by following the rules using the function isSafe( ):
    a. No two Queens share the same column
    b. No two Queens share the same diagonal
    c. No two Queens share the same row
5. Display the matrix with all possible solutions.
6. Stop.

Program:

```
def isSafe(mat, r, c):
    # return false if two queens share the same column
    for i in range(r):
        if (mat[i][c] == 'Q'):
            return False
    # return false if two queens share the same row
    (i, j) = (r, c)
    while (i >= 0 and j >= 0):
        if (mat[i][j] == 'Q'):
            return False
        i = i - 1
        j = j - 1
    # return false if two queens share the same diagonal
```

```
    (i, j) = (r, c)
    while (i >= 0 and j < len(mat)):
        if (mat[i][j] == 'Q'):
            return False
        i = i - 1
        j = j + 1
    return True
def printSolution(mat):
    for r in mat:
        print(str(r).replace(',', '').replace('\'', ''))
    print( )
def nQueen(mat, r):
    if (r == len(mat)):
        printSolution(mat)
            return
    for i in range(len(mat)):
        if (isSafe(mat, r, i)):
            mat[r][i] = 'Q'
            nQueen(mat, r + 1)
            mat[r][i] = '–'
N = int(input("Enter the number of Queens to be placed:" ))
mat = [['–' for x in range(N)] for y in range(N)]
print("The possible solutions are:")
nQueen(mat, 0)
```

Result:

Thus, the above program to implement N-Queen problem using Backtracking method was done and executed successfully.

Ex No: 8 IMPLEMENTATION OF TRAVELLING SALESMAN PROBLEM USING BRANCH AND BOUND

Aim:

To write a python program to implement Travelling Salesman problem using Branch and Bound method.

Algorithm:

1. Start
2. Get the number of cities to be visited and the cost Matrix as input.
3. Find cost of reduced matrix. Cost is given by summation of subtracted amount from the cost matrix to convert it in to reduce matrix.
4. Prepare state space tree for the reduce matrix
5. Find least cost valued node by computing reduced cost node matrix with every remaining node.
6. Compute the cost of newly created reduced matrix.
7. Stop.

Program:

```
import numpy as np

from sys import maxsize

from itertools import permutations


def TSProblem(graph, s):

   vertex = []

   for i in range(V):

      if (i != s):

         vertex.append(i)

   min_path = maxsize

   next_permutation=permutations(vertex)

   for i in next_permutation:
```

```
        current_pathweight = 0
        k = s
        for j in i:
            current_pathweight += graph[k][j]
            k = j
        current_pathweight += graph[k][s]
        min_path = min(min_path, current_pathweight)
    return min_path


V=int(input("Enter the number of cities: "))
print("Enter the cost matrix: ")
matrix = [[int(input()) for c in range (V)] for r in range(V)]
matrix = np.array(matrix).reshape(V,V)
print("The Cost Matrix is\n",matrix)
s = 0
cost=TSProblem(matrix, s)
print("Travelling Salesman Cost is: ", cost)
```

Result:

Thus, the above program to implement Travelling Salesman problem using Branch and Bound was done and executed successfully.